Plaza Works App - Automated Test Suite

Instructions for Replit

Paste the prompt at the bottom of this document into Replit to have it run all tests automatically.

Test Configuration

Test User Credentials

```
const TEST_USERS = {
  owner: {
    email: 'owner@test.plazaworks.com',
    password: 'TestOwner123!',
    role: 'Business Owner',
    employmentType: 'Permanent'
  },
  plumber1: {
    email: 'plumber1@test.plazaworks.com',
    password: 'TestPlumber123!',
    role: 'Plumber',
    employmentType: 'Permanent'
  },
  plumber2: {
    email: 'plumber2@test.plazaworks.com',
    password: 'TestPlumber456!',
    role: 'Plumber',
    employmentType: 'Contractor'
  },
  apprentice: {
    email: 'apprentice@test.plazaworks.com',
    password: 'TestApprentice123!',
    role: 'Apprentice',
    employmentType: 'Permanent'
  },
  tiler: {
    email: 'tiler@test.plazaworks.com',
    password: 'TestTiler123!',
    role: 'Tiler',
    employmentType: 'Contractor'
  }
```

```
};

const TEST_CLIENT = {
  name: 'Test Client Pty Ltd',
  contactName: 'John Smith',
  email: 'client@test.plazaworks.com',
  phone: '0400000000',
  address: '123 Test Street, Brisbane QLD 4000'
};

const TEST_JOB = {
  type: 'Drainage',
  description: 'Test drainage job - blocked stormwater pit',
  address: '123 Test Street, Brisbane QLD 4000',
  quotedDuration: 1.5,
  quotedLabor: 2955,
  quotedMaterials: 2993
};
```

Test Suites

Suite 1: Authentication Tests

```
describe('Authentication', () => {
  test('1.1.1 - Sign up new user', async () => {
    // Navigate to signup
    // Enter valid email and password
    // Submit form
    // Verify redirect to dashboard
    // Verify user record created in database
  });

  test('1.1.2 - Log in with valid credentials', async () => {
    // Navigate to login
    // Enter TEST_USERS.owner credentials
    // Submit form
    // Verify successful login
    // Verify dashboard loads
  });

  test('1.1.3 - Log out', async () => {
    // Log in first
    // Click logout button
    // Verify redirect to login page
    // Verify session cleared
```

```
  });

  test('1.1.4 - Reject invalid credentials', async () => {
    // Navigate to login
    // Enter valid email with wrong password
    // Submit form
    // Verify error message displayed
    // Verify still on login page
  });

  test('1.1.5 - Password reset flow', async () => {
    // Navigate to forgot password
    // Enter valid email
    // Submit
    // Verify reset email sent (check logs or mock)
  });
});
```

Suite 2: User Management Tests

```
describe('User Management', () => {
  test('1.2.1 - Create new user as owner', async () => {
    // Log in as owner
    // Navigate to team/users
    // Click add user
    // Fill form with test data
    // Submit
    // Verify user appears in list
  });

  test('1.2.2 - Assign role to user', async () => {
    // Log in as owner
    // Edit existing user
    // Change role dropdown
    // Save
    // Verify role updated in database
  });

  test('1.2.3 - Set employment type', async () => {
    // Log in as owner
    // Edit user
    // Set employment type to Contractor
    // Save
    // Verify type saved
  });

  test('1.2.4 - Assign custom permissions', async () => {
```

```
    // Log in as owner
    // Edit user
    // Toggle specific permissions independent of role
    // Save
    // Verify permissions saved correctly
  });

  test('1.2.5 - List all users', async () => {
    // Log in as owner
    // Navigate to team/users
    // Verify all test users displayed
    // Verify roles shown correctly
  });
});
```

Suite 3: Role-Based Access Tests

```
describe('Role-Based Access', () => {
  test('1.3.1 - Owner has full access', async () => {
    // Log in as owner
    // Verify can access: dashboard, jobs, scheduling, quotes, invoices, team, vehicles,
settings
  });

  test('1.3.2 - Plumber has limited access', async () => {
    // Log in as plumber1
    // Verify can access: own dashboard, assigned jobs, own schedule
    // Verify cannot access: user management, all financials, other users data
  });

  test('1.3.3 - Apprentice has minimal access', async () => {
    // Log in as apprentice
    // Verify can access: own dashboard, assigned jobs
    // Verify cannot access: job creation, quotes, invoices, team management
  });

  test('1.3.4 - Contractor cannot see sensitive data', async () => {
    // Log in as plumber2 (contractor)
    // Verify cannot see: financial reports, other contractor details, profit margins
  });
});
```

Suite 4: Job Management Tests

```
describe('Job Management', () => {
  test('1.4.1 - Create new job', async () => {
    // Log in as owner
```

```
  // Navigate to jobs
  // Click new job
  // Fill: client, address, type, description
  // Submit
  // Verify job created with status "New"
});

test('1.4.2 - View job list', async () => {
  // Log in as owner
  // Navigate to jobs
  // Verify list displays with columns: status, client, date, type
});

test('1.4.3 - Filter jobs by status', async () => {
  // Create jobs with different statuses
  // Apply status filter
  // Verify only matching jobs shown
});

test('1.4.4 - Edit job details', async () => {
  // Open existing job
  // Change description
  // Save
  // Verify changes persisted
});

test('1.4.5 - Change job status', async () => {
  // Open job
  // Change status from New to In Progress
  // Save
  // Verify status updated in list
});

test('1.4.6 - Assign tradesman to job', async () => {
  // Open job
  // Select tradesman from dropdown
  // Save
  // Verify assignment saved
});

test('1.4.7 - Assign apprentice to job', async () => {
  // Open job with tradesman assigned
  // Add apprentice
  // Save
  // Verify both assigned
});

test('1.4.8 - View job details', async () => {
```

```
    // Click on job in list
    // Verify detail view shows: all fields, history, assignments, linked quotes/invoices
  });
});
```

Suite 5: Scheduling Tests

```
describe('Scheduling', () => {
  test('1.5.1 - View calendar', async () => {
    // Navigate to schedule
    // Verify calendar renders
    // Verify existing scheduled jobs visible
  });

  test('1.5.2 - Schedule job on date', async () => {
    // Click date on calendar
    // Select job
    // Assign tradesman
    // Save
    // Verify job appears on calendar
  });

  test('1.5.3 - Create multi-day job', async () => {
    // Schedule job spanning 3 days
    // Verify job shows on all 3 days
  });

  test('1.5.4 - Filter calendar by tradesman', async () => {
    // Select tradesman filter
    // Verify only their jobs shown
  });

  test('1.5.5 - Reschedule job', async () => {
    // Move scheduled job to different date
    // Verify old date cleared
    // Verify new date shows job
  });

  test('1.5.6 - Tradesman sees own schedule', async () => {
    // Log in as plumber1
    // View schedule
    // Verify only own assigned jobs visible
  });
});
```

Suite 6: Client Portal Tests

```
describe('Client Portal', () => {
  test('2.1.1 - Client can log in', async () => {
    // Navigate to client portal URL
    // Log in with client credentials
    // Verify portal dashboard loads
  });

  test('2.1.2 - Client sees only own jobs', async () => {
    // Log in as client
    // View jobs list
    // Verify only jobs for this client shown
  });

  test('2.1.3 - Client cannot access other clients', async () => {
    // Attempt to access job belonging to different client
    // Verify access denied
  });

  test('2.1.4 - Client views job status', async () => {
    // Open job in portal
    // Verify status, timeline, milestones visible
  });

  test('2.1.5 - Client views scheduled dates', async () => {
    // Open job with scheduled visits
    // Verify dates displayed
  });
});
```

Suite 7: PC Items Tests

```
describe('PC Items', () => {
  test('2.2.1 - Add PC items to job', async () => {
    // Log in as staff
    // Edit job
    // Add PC items list
    // Save
    // Verify items attached
  });

  test('2.2.2 - Client views PC items', async () => {
    // Log in as client
    // View job with PC items
    // Verify list displayed with selection options
  });

  test('2.2.3 - Client selects items', async () => {
```

```
    // Make selections
    // Save
    // Verify selections stored
  });

  test('2.2.4 - Client revises selections', async () => {
    // Change previous selections
    // Save
    // Verify updated selections stored
  });

  test('2.2.5 - Client submits for approval', async () => {
    // Click submit/confirm
    // Verify status changes to submitted
  });

  test('2.2.6 - Staff sees client selections', async () => {
    // Log in as staff
    // View job
    // Verify client selections visible
  });

  test('2.2.7 - Staff approves selections', async () => {
    // Approve selections
    // Verify status changes to approved
  });
});
```

Suite 8: Notifications Tests

```
describe('Notifications', () => {
  test('2.5.1 - Job status change triggers notification', async () => {
    // Change job status
    // Verify notification created for client
  });

  test('2.5.2 - Upcoming visit reminder sent', async () => {
    // Schedule job for tomorrow
    // Run reminder job (or verify it's queued)
    // Verify reminder notification created
  });

  test('2.5.3 - Staff assignment notification', async () => {
    // Assign tradesman to job
    // Verify tradesman receives notification
  });
```

```
  test('2.5.4 - In-app notifications display', async () => {
    // Log in
    // Check notification inbox/bell
    // Verify recent notifications shown
  });
});
```

Suite 9: Quote Tests

```
describe('Quotes', () => {
  test('3.1.1 - Create quote', async () => {
    // Navigate to quotes
    // Click new
    // Fill details and line items
    // Save
    // Verify quote created
  });

  test('3.1.2 - Add line items', async () => {
    // Add labor line
    // Add materials line
    // Add subcontractor line
    // Verify all items displayed with amounts
  });

  test('3.1.3 - Quote total calculates correctly', async () => {
    // Add multiple line items
    // Verify subtotal correct
    // Verify GST calculated (10%)
    // Verify total correct
  });

  test('3.1.4 - Link quote to job', async () => {
    // Create quote
    // Select existing job
    // Verify link established
  });

  test('3.1.5 - Send quote to client', async () => {
    // Click send
    // Enter client email
    // Submit
    // Verify quote status changes to sent
  });

  test('3.1.6 - Client can view quote', async () => {
    // Access quote link as client
```

```
    // Verify quote details displayed
  });

  test('3.1.7 - Client accepts quote', async () => {
    // Click accept
    // Verify status changes to accepted
  });

  test('3.1.8 - Client declines quote', async () => {
    // Create another quote
    // Click decline
    // Verify status changes to declined
  });
});
```

Suite 10: Invoice Tests

```
describe('Invoices', () => {
  test('3.2.1 - Create invoice', async () => {
    // Navigate to invoices
    // Click new
    // Fill details
    // Save
    // Verify invoice created
  });

  test('3.2.2 - Convert quote to invoice', async () => {
    // Open accepted quote
    // Click convert to invoice
    // Verify invoice pre-filled from quote
  });

  test('3.2.3 - Create deposit invoice', async () => {
    // Create invoice for deposit amount only
    // Verify correct amount
  });

  test('3.2.4 - Create progress invoice', async () => {
    // Create partial invoice
    // Verify remaining balance tracked
  });

  test('3.2.5 - Invoice total correct', async () => {
    // Create invoice with multiple lines
    // Verify subtotal, GST, total all correct
  });
});
```

Suite 11: Payment Tests

```
describe('Payments', () => {
  test('3.3.1 - Generate payment link', async () => {
    // Open invoice
    // Click create payment link
    // Verify short link generated
  });

  test('3.3.2 - Payment link loads payment page', async () => {
    // Open payment link
    // Verify payment page loads
  });

  test('3.3.3 - Record payment received', async () => {
    // Mark invoice as paid
    // Verify status changes
    // Verify payment record created
  });

  test('3.3.4 - Partial payment tracked', async () => {
    // Record payment less than total
    // Verify balance remaining displayed
  });

  test('3.3.5 - Deposit unlocks scheduling', async () => {
    // Pay deposit invoice
    // Verify job scheduling becomes available
  });
});
```

Suite 12: Vehicle Tests

```
describe('Vehicles', () => {
  test('4.1.1 - Add vehicle', async () => {
    // Navigate to vehicles
    // Click add
    // Enter: rego, make, model
    // Save
    // Verify vehicle created
  });

  test('4.1.2 - Assign driver', async () => {
    // Edit vehicle
    // Assign to user
    // Save
```

```
    // Verify assignment
  });

  test('4.1.3 - View vehicle list', async () => {
    // Navigate to vehicles
    // Verify all vehicles displayed with drivers
  });

  test('4.1.4 - Log vehicle usage', async () => {
    // Select vehicle
    // Add usage log
    // Verify log saved
  });

  test('4.1.5 - Set service schedule', async () => {
    // Edit vehicle
    // Set next service date
    // Save
    // Verify date saved
  });

  test('4.1.6 - Service overdue alert', async () => {
    // Set service date to past
    // Verify overdue indicator shown
  });

  test('4.1.7 - Record service completed', async () => {
    // Mark service done
    // Verify history updated
    // Verify next service date cleared or updated
  });
});
```

Suite 13: Checklist Tests

```
describe('Checklists', () => {
  test('4.3.1 - View job checklist', async () => {
    // Open job with checklist
    // Verify checklist items displayed
  });

  test('4.3.2 - Complete checklist item', async () => {
    // Tick item as done
    // Verify marked complete with timestamp
  });

  test('4.3.3 - Checklist progress shown', async () => {
```

```
    // Complete some items
    // Verify progress indicator (e.g., 3/10)
  });

  test('4.3.4 - All items complete', async () => {
    // Complete all items
    // Verify checklist shows finished
  });
});
```

Suite 14: Photo Tests

```
describe('Job Photos', () => {
  test('4.4.1 - Upload single photo', async () => {
    // Open job
    // Click add photo
    // Select test image file
    // Verify upload succeeds
    // Verify photo displays
  });

  test('4.4.2 - Upload multiple photos', async () => {
    // Upload 5 photos
    // Verify all appear in gallery
  });

  test('4.4.3 - View photo gallery', async () => {
    // Open job with photos
    // Verify gallery view works
  });

  test('4.4.4 - Add photo caption', async () => {
    // Add caption to photo
    // Save
    // Verify caption displayed
  });

  test('4.4.5 - Client sees photos in portal', async () => {
    // Log in as client
    // View job
    // Verify photos visible
  });

  test('4.4.6 - Delete photo', async () => {
    // Remove a photo
    // Verify deleted
  });
```

```
});
```

Suite 15: Time Tracking Tests

```
describe('Time Tracking', () => {
  test('5.1.1 - Log hours on job', async () => {
    // Open job
    // Enter hours worked
    // Save
    // Verify hours saved
  });

  test('5.1.2 - Log hours with specific date', async () => {
    // Enter hours for yesterday
    // Verify logged against correct date
  });

  test('5.1.3 - View daily hours', async () => {
    // Check today's hours
    // Verify correct total
  });

  test('5.1.4 - View weekly hours', async () => {
    // Check this week's hours
    // Verify correct total
  });

  test('5.1.5 - Edit logged hours', async () => {
    // Change previous entry
    // Verify update saved
  });

  test('5.1.6 - Hours shown on job', async () => {
    // View job detail
    // Verify all logged hours displayed
  });
});
```

Suite 16: Profitability Tests

```
describe('Profitability', () => {
  test('5.2.1 - Job cost calculation', async () => {
    // View completed job with hours logged
    // Verify labor cost = hours x hourly rate
  });

  test('5.2.2 - Job profit displayed', async () => {
```

```
    // View job with revenue and costs
    // Verify profit = revenue - costs
  });

  test('5.2.3 - Duration variance shown', async () => {
    // View job that took longer than quoted
    // Verify variance displayed
  });
});
```

Suite 17: Capacity Tests

```
describe('Capacity Planning', () => {
  test('5.4.1 - View capacity dashboard', async () => {
    // Navigate to capacity page
    // Verify % booked shown for coming weeks
  });

  test('5.4.2 - Capacity calculates correctly', async () => {
    // Schedule known number of days
    // Verify percentage is accurate
  });

  test('5.4.3 - Per tradesman view', async () => {
    // Filter by tradesman
    // Verify individual availability shown
  });
});
```

Suite 18: Integration Tests

```
describe('Integration Tests', () => {
  test('INT-1 - Full job lifecycle', async () => {
    // Create job
    // Create and send quote
    // Accept quote as client
    // Create deposit invoice
    // Mark deposit paid
    // Schedule job
    // Log hours
    // Upload photos
    // Complete checklist
    // Mark job complete
    // Create final invoice
    // Mark paid
    // Verify job flows through all stages
  });
```

```
  test('INT-2 - Client portal complete flow', async () => {
    // Client accepts terms
    // Client pays deposit
    // Client selects PC items
    // Client views progress updates
    // Client sees photos
    // Verify all touchpoints work
  });

  test('INT-3 - Quote to job connection', async () => {
    // Create quote
    // Accept quote
    // Verify job linked
    // Verify job unlocks after deposit
  });

  test('INT-4 - Time affects profitability', async () => {
    // Log hours on job
    // Verify profitability updates
  });

  test('INT-5 - Schedule affects capacity', async () => {
    // Schedule multiple jobs
    // Verify capacity dashboard reflects bookings
  });

  test('INT-6 - Permissions respected throughout', async () => {
    // Test each role accessing various features
    // Verify access control enforced
  });
});
```

Suite 19: API Tests

```
describe('API Endpoints', () => {
  test('API-1 - GET /api/v1/jobs returns jobs', async () => {
    // Call endpoint with auth
    // Verify 200 response
    // Verify data structure correct
  });

  test('API-2 - POST /api/v1/jobs creates job', async () => {
    // POST valid job data
    // Verify 201 response
    // Verify job created
  });
```

```
  test('API-3 - Unauthorized request rejected', async () => {
    // Call endpoint without auth
    // Verify 401 response
  });

  test('API-4 - Invalid data rejected', async () => {
    // POST invalid job data
    // Verify 400 response with error message
  });
});
```

Replit Prompt

Copy and paste this entire prompt into Replit:

I need you to run a comprehensive automated test suite for the Plaza Works Job Management App.

SETUP FIRST:
1. Install testing dependencies if not present (Jest, Playwright, or your preferred test framework)
2. Create a test database or use test mode
3. Create these test users in the database:
   - owner@test.plazaworks.com (Business Owner, Permanent)
   - plumber1@test.plazaworks.com (Plumber, Permanent)
   - plumber2@test.plazaworks.com (Plumber, Contractor)
   - apprentice@test.plazaworks.com (Apprentice, Permanent)
   - tiler@test.plazaworks.com (Tiler, Contractor)
   All with password: Test123!
4. Create a test client: Test Client Pty Ltd, client@test.plazaworks.com

TEST SUITES TO RUN:

PHASE 1 - Foundation:
- Authentication (signup, login, logout, invalid login, password reset)
- User Management (create, assign role, set employment type, custom permissions, list users)
- Role-Based Access (owner full access, plumber limited, apprentice minimal, contractor restricted)
- Job Management (create, list, filter, edit, change status, assign tradesman, assign apprentice, view details)
- Scheduling (view calendar, schedule job, multi-day job, filter by tradesman, reschedule, tradesman own view)

PHASE 2 - Client Experience:
- Client Portal (login, see own jobs only, cannot access other clients, view status, view dates)
- PC Items (add to job, client views, client selects, client revises, submit approval, staff sees, staff approves)
- Notifications (status change, visit reminder, assignment, in-app display)

PHASE 3 - Financial:
- Quotes (create, line items, total calculates, link to job, send, client view, accept, decline)
- Invoices (create, from quote, deposit, progress, final, total correct)
- Payments (generate link, link works, record payment, partial payment, deposit unlocks scheduling)

PHASE 4 - Operations:
- Vehicles (add, assign driver, list, log usage, service schedule, overdue alert, service complete)
- Checklists (view, complete item, progress, all complete)
- Photos (upload single, upload multiple, gallery, caption, client sees, delete)

PHASE 5 - Intelligence:
- Time Tracking (log hours, log with date, daily total, weekly total, edit hours, hours on job)
- Profitability (job cost calc, profit display, duration variance)
- Capacity (view dashboard, calculation accuracy, per tradesman view)

INTEGRATION TESTS:
- Full job lifecycle from creation to final payment
- Complete client portal flow
- Quote to job connection
- Time affects profitability
- Schedule affects capacity
- Permissions respected throughout

API TESTS:
- GET endpoints return correct data
- POST endpoints create records
- Unauthorized requests rejected
- Invalid data rejected with proper errors

FOR EACH TEST:
1. Run the test
2. Record PASS or FAIL
3. If FAIL, capture the error message
4. Continue to next test

AFTER ALL TESTS:
1. Generate a test report showing:
   - Total tests run
   - Tests passed
   - Tests failed

  - Pass rate percentage
  - List of all failures with error details
2. For any failures, suggest fixes
3. Show me the complete test report

Please run all tests now and show me the results.

Alternative: Simpler Manual Testing Prompt

If Replit struggles with the full automated suite, use this simpler prompt:

Please test the Plaza Works Job Management App by manually verifying each feature works.

Create test data:
- Test user: owner@test.plazaworks.com / Test123!
- Test client: Test Client Pty Ltd

Then test these critical paths and report results:

1. AUTHENTICATION: Can you sign up, log in, and log out?
2. CREATE JOB: Can you create a new job with client, address, and type?
3. SCHEDULING: Can you schedule the job on the calendar?
4. QUOTE: Can you create a quote with line items and correct totals?
5. INVOICE: Can you create an invoice from the quote?
6. PAYMENT LINK: Can you generate a payment link?
7. VEHICLE: Can you add a vehicle and assign a driver?
8. PHOTO UPLOAD: Can you upload a photo to a job?
9. TIME LOGGING: Can you log hours on a job?
10. CAPACITY VIEW: Can you see the capacity/booking percentage?

For each test, tell me:
- What you tried
- Whether it worked (PASS/FAIL)
- Any errors you encountered

Then summarize what's working and what needs fixing.